



关于作者



wkGCaSS

发动~暴~走~魔法阵~

回答
452

文章
20

关注者
12,251

关注他

发私信

libnginx | 把nginx当作http server/client库使用



wkGCaSS

发动~暴~走~魔法阵~

关注他

收录于 · wkgcass的网络专栏 >

75 人赞同了该文章 >

在写一个http server时，我们通常会选择java、go等带GC的语言，或者rust这样的高级语言，而这些语言通常会在语言本身的标准库之上构建一套http服务。

即使因为性能或功能原因，通过ffi调用底层库，也基本仅限于网络处理层面，而非上层应用。例如netty，只有薄薄的一层fd+epoll封装和openssl封装，上层都是用java构建的。

用C语言编写http server的应用场景不多，由于C的语法单一、没有RAII，所以通常需要借助脚本语言才能把负担降低到一个合理的程度。比如openresty，它在nginx之上使用lua语言进行开发。

C语言本身编写http服务不顺手，所以需要扩展nginx，在其上增加其他语言的支持。

把这个思路反过来：我们为什么不直接把nginx拿过来，放到别的语言里作为库使用呢？

众所周知，nginx是全世界范围使用最广泛的http服务器和负载均衡，它性能优异、延迟低、资源占用少，同时具有很强的扩展性。编译和运行nginx也非常方便，仅需要openssl和pcre这样的常见依赖。

那么为什么目前没有“将nginx作为库”的成熟方案呢？我想有如下两个阻碍：

1. 【技术方面】nginx的进程模型以及其广泛使用全局变量的编码习惯，决定了它原生无法适配多线程的服务器模型
2. 【选型方面】需要动态生成内容的http服务器，通常瓶颈不在于计算性能，不需要使用底层语言。不如使用java或go这样带GC的语言，降低负担

接下来我将分析如何解决多线程问题，以及为什么要使用nginx作为http库。

解决线程模型问题



nginx除了广泛使用的[master-worker模型](#)⁺外，还自带一个“单进程”模型：`daemon off;`
`master_process off;`。初衷是为了方便程序员开发调试使用。源码中可以看到，除了在
`event.c`里面存在一些微小差异外，上层处理逻辑中，`master-worker`模型和[单进程模型](#)⁺没
有任何区别。

单进程是nginx最简单的模型，我们就从这里入手。

将nginx的main函数进行少许改造，将 `main` 改成 `ngx_lib_main`，使用if宏移除
`daemon`、`pid` 文件、`stderr` 重定向、多余的信号处理等功能，并且强制使用单进程
`ngx_single_process_cycle` 启动，然后稍微改造一下构建脚本，添加 `-fPIC` 和 `-shared`，
我们就得到了 `libnginx.so`。

通过这种方式，我们可以完美适配所有静态链入nginx的模块，比如h2,h3,ssl等，官方库中
大部分模块都是静态链入的。如果静态模块存在一些动态库依赖也没关系（比如openssl就
是动态库），只要模块本身是静态的就好。

无论是用 `dlopen`⁺ 加载，还是编译时直接链接，我们都可以轻松的在自己程序的某个线程
上，启动一个nginx服务器。

但是如果在第二个线程上直接启动第二个nginx服务器，则会数据错乱，或者进程直接挂
掉。

这是意料之内的。nginx大量使用全局变量，在第二个nginx worker启动时，本该由第一个
worker使用的数据会被覆盖，导致程序异常。

那么有什么方法，可以不做整体修改就实现多线程模型呢？

答案就是：私有符号以及 `RTLD_LOCAL`⁺。

当我们使用`dlopen`加载一个动态链接库时，我们可以指定一个flag：`RTLD_LOCAL`，将它的
符号与其他动态链接库隔离开。比方说有两个动态链接库，`a.so` 和 `b.so`，它们都提供了一个
名为 `hello` 的函数，但实现内容不同。我们使用 `RTLD_LOCAL` 分别加载这两个动态库，使用
`dlsym` 分别获取并调用这两个 `hello` 函数，两次调用会产生不同的结果。

照着这个思路，我们可以将 `libnginx.so` 复制很多份，每个线程一份，然后分别加载这些动
态链接库，再在各自的线程上分别调用它们的 `ngx_lib_main` 函数。这样，每个线程都拥有
自己的全局变量，自然就不会相互影响了。

每个worker线程都会使用相同的监听端口，这个问题也很好解决，使用[reuseport](#)⁺就可以
了。就如同任何正常的使用reuseport的程序一样，让内核帮我们将连接分发到各个线程
去。

由于nginx本身存在大量api，全部暴露可能产生误用。我们可以使用version-script将不需
要的符号隐藏，不想让用户使用的“内部”api可以得到保护。

我们可以让 `libnginx.so` 仅暴露一个入口函数 `libngx()`，用于获取api table结构体指针，
所有api都通过函数指针暴露。

在linux下使用这样的version-script：

```
libnginx {  
    global:  
        libngx;  
}
```

```
local:
    *;
};
```

在 macos 下使用这样的 exported-symbols:

```
_libngx
```

到此，我们解决了“让 nginx 在多个线程上跑起来”的问题。

其实还有一个思路是：强行让用户使用多进程模型。但是这就大大限制了应用场景。rust 可能相对还好一点（但是一般也不会考虑多进程），但是对于 java 和 go，它们是有运行时的，不能在代码里随意调 fork。

目前绝大多数开发者并不习惯跨进程操作，即使真的跑起来，多进程库的受众也一定很小。

这也是为什么不仅仅做一个 nginx 模块，而是要做一个库。nginx 模块的限制比多进程库的限制更大，如果仅通过编写模块扩展 nginx，那基本就和 java、go 等语言说再见了。

为什么使用 nginx 构建 http 服务？

现在 java、go、rust 等语言的 http 库已经有着广泛应用，为什么要再基于 nginx 做一个库呢？是否有什么场景非常适合这么做呢？

经验复用

由于基础代码就是 nginx 本身，所以原本在 nginx 上积累的经验可以直接拿来使用。而在使用 libnginx 库积攒的经验，也可以反馈到普通版本的 nginx 上。

简单来说，相比引入其他 http server（例如 [lighttpd](#)⁺ 等），使用 nginx 更容易实现经验复用。

除了老生常谈的各种 nginx 参数以及绑核绑中断外，还有一些 nginx 相关的特殊用法。比如 [f-stack](#) 官方支持 nginx 1.25.2。有了 libnginx，就可以让自己的 http 服务运行于 [dpdk](#)⁺ 之上。

目前我已经完成了 [f-stack](#) 相关的适配，不过这就是另一篇文章了。

TLS⁺、GZip⁺等功能

对于 java/go 等语言编写的内网网关，性能堵点常在于系统调用，所以语言本身运行时的性能以及 gc 开销会被缩小。但是对接公网的网关服务通常需要 tls 加密、并且需要启用 gzip 减少带宽消耗。通常 tls 的 cpu 消耗会占到网关服务的 70% 甚至更多，这也导致 java/go 等语言提供的服务，很少有直接通过 tls 暴露的，大多需要走 nginx/haproxy 等软件做一次转发。

比如把 **自动签发证书** 作为宣传口号的 [Caddy](#)，在与 nginx 的 https 请求性能测试对比中，差距非常明显。

TLS和gzip 运算还可以被硬件加速卡加速，例如 [Intel QAT⁺](#)。Intel 提供了 nginx 的 patch，而其他不支持 `async ssl engine` 的软件或框架，就得自己动手适配才能接入了。

轻便、易扩展、插件多、集成网关功能

nginx 的资源占用、扩展性、社区插件数量是毋庸置疑的多。日志、监控也都有现成的解决方案，有需求则可无缝搬过来使用。这些都无需赘述了。

同时 nginx 还有负载均衡能力，可以作为网关使用。

比如一个 API 网关，你可能需要先验证 `access-key` 和签名，然后才能转发给后端服务，而验证签名又需要完整 `payload`。这时如果依赖远端服务，则 100% 的请求流量都需要经过网络完整的复制一份。

我们考虑可以让 nginx 直接变成这个鉴权服务，在看完完整的请求内容后，再决定是否要转发给 `upstream`。nginx 直接挂在 4 层 LB 后面就行。

Rust 和 Swift

上面提到了三种编程语言：java、go、rust，这也是目前服务端最火的三种语言，分别有各自的应用场景。对于 java 和 go，它们虽然可以使用 `libnginx`，但并不能 100% 的发挥 nginx 的性能。因为当 http 请求到来时，库一定会“回调”处理函数，那么这就属于不同语言之间的“upcall”。

对于 go 来说无论 `downcall` 还是 `upcall` 都非常耗性能；对于 java 来说，`upcall` 开销也远超普通的 java 内部函数调用。

而对于 rust 则没有这方面的顾虑。`upcall` 也只是一次普通的函数调用而已。

正因为有了内存安全的高级语言，同时没有额外 `upcall` 开销，可以让 `libnginx` 的使用面进一步扩大。即使是 `upcall` 开销较大的 java，打赢阻塞模式的 `servlet` 应该也不成问题。

http server 常常会有很重的业务逻辑，用 C 编写 json 序列化反序列化、数据库操作、消息队列、配置中心等功能几乎是不可能的。而 rust 就没有这方面的担忧，上述功能可以很方便的用 rust 实现。

除了 rust 外，还有一个非常有潜力的语言：**Swift**。

你可以把 Swift 当成“有 `class` 的、自带引用计数的、有完整值类型和泛型的、支持函数式编程的 C”。它支持无缝调用 C 代码，也可以无缝编写传给 C 回调的函数，中途不需要任何形式的 `binding`，编译器可以自动处理 Swift 和 C 的混编。后端自然也是使用苹果的 LLVM。

因为它使用自动引用计数，所以对开发者的心智负担远低于 rust。

大多数情况你都可以不用考虑对象管理。在需要极限性能的简单场景，你可以栈上分配结构体并使用 `inout` 传递或者使用 `non-Copyable` 结构体；需要极限性能的复杂场景，你也可以全程手动管理内存。

IBM 和 Swift 社区很早就推 [Server Side Swift⁺](#)，但是一直没有什么起色，后来 IBM 退出了，社区剩下 [vapor⁺](#) 这个可堪一用的框架。

但是 `vapor` 有两个问题。

一是依赖太重。使用 vapor，即使是一个简单的 hello world，也相当于使用了几十个库。而 swift 作为一个支持继承、多态、protocol 的 aot 语言，编译自然是比较耗时的。

比如我正在用 swift 写一套用户态协议栈，本来开最高级别（-Ounchecked+cmo+wmo）的全量编译也只在 2 分钟内，想着 swift 毕竟是个高级语言，还是用 http 提供接口比较好。项目里加上 vapor 之后，一次最高级别的全量编译竟然要 10 分钟以上。

这个编译级别会做跨模块的内联，比如 A 模块的一些 inline 函数修改后，可能会影响 B 模块，这就导致每次改到带 inline 的公共模块时，都要做一次全量编译。之前全量也就 2 分钟，我觉得不是什么问题，引入 vapor 之后一次全量就要十几分钟，开发体验直线下降。

另一个问题时，server side swift 目前生态不足，除了 vapor 还算稳定之外，其他库大多是玩票性质的，也就是能跑的程度。这也反过来限制了 vapor 的发挥。毕竟服务端内存不值钱，那我写个 http 服务，为啥不用 java 呢，还不用担心哪天写出循环依赖；什么 kafka、es、各种协议的解析、加解密，全都是现成的 maven 库。

所以 Server Side Swift 目前暂时并不适合用于写业务。

而加上 nginx 就不一样了。如果写网关服务，swift 有非常完善的类型系统，比 lua 更适合规模大的项目，复杂场景性能会远高于 luajit，最差也不会比 luajit 更差，并且不走 tracing GC，不会出现意料之外的卡顿。

另一方面，对于我所需的“简单 http server/client”的场景，libnginx 也可以提供“零额外依赖”、极短的编译时间、以及久经考验的稳定性。

改造 nginx

nginx 本身就是一个易于扩展的框架。改造成库后，最直接的调用方法，自然也是利用它已有的扩展机制。

http 库要实现的最基础的内容，就是**接收一个带 body 的 http 请求，然后返回一个带 body 的 http 响应**。我们就从这里入手。

处理 http 请求

想要处理 http 请求、发送 http 响应，我们需要编写一个 http module。我们将模块本体编写好，作为 lib 的一部分。这样，用户就只需要填充必要的回调函数即可。

在使用时，我们需要给 nginx 注册一个 CONTENT_PHASE 的回调，不妨称作 req_handler。与绝大多数异步库一样，如果要获取请求 body，还需在 req_handler 的内部，给 req 提供一个 body_handler。

无论是在 req_handler 还是 body_handler 中，我们都可以把任务交给其他线程处理（需要返回 NGX_AGAIN 告诉 nginx 请求尚未处理完成），等处理完成后，再把结果传给 nginx worker 的线程并操作 req 发送结果。

为了能够把消息传递给 worker，我们需要引入一个无锁 mpsc 队列：由任意线程生产，由 worker 消费。其他线程将事件推入队列时，还需调用 ngx_notify 激活事件循环。

为了处理队列中的事件，我们还需要增加一个回调函数，在 nginx 的每个循环中都调用一次。

既然这个函数每个循环都要调用一次，那我们不妨再让这个函数返回一个“时间”，用作 epoll/kevent 等待的最大时间。这样，我们甚至可以在这个框架下构造自己的定时任务系统。

众所周知，nginx可以配置多个 server 块，每个 server 块中又可以配置多个 location 块。

我们需要一个机制，告诉 req_handler，这个req属于哪个 location，这样我们的 libnginx就能利用nginx配置文件的强大功能了。

最简单的方法，就是在配置文件的location块中，使用自定义参数指定一个**数字id**。在 req_handler中获取这个id，并编写与之匹配的处理逻辑。

我们不妨把这个参数取名为 upcall {id}。

```
listen 0.0.0.0:7788 reuseport;
http2 on;
location / {
    upcall 1;
}
location = /sample {
    upcall 1;
}
location = /async {
    upcall 2;
}
location = /echo {
    upcall 3;
}
location = /storage {
    if ($request_method = GET) {
        upcall 4; break;
    }
    if ($request_method = POST) {
        upcall 5; break;
    }
    return 405;
}
```

知乎 @wkGCaSS

nginx 配置，upcall可以与rewrite模块混合使用

除去 location 块外，该参数还可以跟nginx rewrite module的 if/break 一同使用。

到此，我们已经完成了请求处理功能的改造。

作为http客户端

一个http库不能只有服务端而没有客户端。

我们都知道，nginx官方提供了一个 auth 模块，可以将请求除了body之外的部分发送至指定server进行鉴权，然后通过响应判断是否认证通过。我们就从这里入手，构造http客户端。

通过用法以及源码可以知道，nginx所谓的“客户端”，其实是构造了一个 subrequest，然后让这个subreq重新穿过nginx的请求处理链，最终通过 proxy 模块将请求发送至远端。

为了构造 subrequest，我们需要一个“main request”，一般来说就是刚刚接收到的请求。但http客户端不应当依赖“当前是否存在请求”这个条件。为了规避nginx的限制，我们还需提供一个方式，手动构造一个 dummy_req，方便用户使用 subrequest。

所以最终http客户端的流程是：

1. 构造 dummy_req
2. 将 dummy_req 作为 main req，创建 subrequest
3. 执行 req
4. nginx应当使用恰当的upstream peer进行subrequest的转发
5. 在回调函数中获取http响应

构造 dummy_req

因为请求是由http连接生成的，连接是由监听生成的，监听是属于某个特定server的；所以，为了构造req，我们首先需要指定这个req由哪个server生成。

这也给了 libnginx 更多可配置能力。每个 server 块都有自己的配置，用户可以根据自己的需求，动态地将 dummy_req 创建在某个特定的 server 块内，这样后续的 subrequest 就可以利用该 server 块的配置进行转发了。

为了方便用户指定server块，我们可以在 server 块中增加一个配置项：server_id \${id}。

```
server {
    server_id 1;
    listen 127.126.125.124:65535 reuseport;
    location / {
        proxy_set_header Connection "";
        proxy_http_version 1.1;
        proxy_pass http://servers/;
    }
}

upstream servers {
    upcall 1;
    keepalive 128;
    keepalive_requests 100000000;
    keepalive_timeout 100000000;
    server 1.1.1.1:1; # template
}
```

知乎 @wkGCaSS

server与upstream的配置

用户在创建 dummy_req 时，指定 server_id，即可让这个 dummy_req 落在指定的 server 块中进行处理。

`dummy_req` 还存在一个问题。`req` 依赖连接，有连接就会有连接 `fd`。而我们的 `dummy_req` 自然是没有 `fd` 的。

为了解决这个问题，我们还需要稍微改造一下 `nginx` 的事件循环：设置一个负数作为 `DUMMY_FD`，事件循环代码中遇到添加/修改/删除 `DUMMY_FD` 的情况，一律不处理，直接返回成功。

`DUMMY_FD` 是负数，它不会跟合法的 `fd` 产生冲突。但是，事件循环通常会以 `fd` 为下标，从数组中存取 `userdata`，`nginx` 的封装也不例外。我们还需要调整这些使用 `fd` 的代码，如果是 `DUMMY_FD` 的话就不执行相关操作。

我们可以特意将 `DUMMY_FD` 设置成一个绝对值非常大的数，比如我用了 `(-1ffffffff)`。这样，如果存在漏改的“使用 `fd` 进行内存访问”的代码，程序会马上挂掉，我们就可以通过 `coredump` 轻松得知哪块代码有问题了。

subrequest

`nginx` 原生的 `subrequest` 仅用作最简单的鉴权使用，所以只支持 `GET` 请求，并且仅会从 `main_req` 中拷贝 `uri` 和 `header`。我们还需修改相关代码，让 `subrequest` 支持自定义 `method`、`uri`、`header`、`body`。

好在 `nginx` 基础代码比较牢靠，这部分改动并没有遇到什么障碍。

动态upstream

从之前的分析中可以得知，`http client` 构造的请求，实际上走的是 `proxy` 流程发出的。一般来说，`upstream` 里的 `server` 列表，是固定在配置文件里面的，不能随意修改。

既然我们要做 `http client`，自然不能直接使用固定的 `server` 地址。所以，我们还需支持“动态 `upstream`”。

“动态 `upstream`”，这个需求更精确一点，其实是“动态 `upstream server`”。我们需要实现一个 `loadbalancer` 模块，当 `nginx` 需要一个 `peer` 进行代理时，会进入我们的模块代码中，此时按需返回 `peer` 给 `nginx` 即可。

由于 `nginx` 的 `upstream` 可以有多个，我们不妨在 `upstream` 块中新增一个配置项 `upcall ${id}`，用来指示当前 `nginx` 想要从哪个 `upstream` 获取 `peer`。

如果将“请求”与“`upstream`”割裂开，`http client` 的使用体验自然不会很好。那我们不妨将需要使用的 `peer` 存放在 `subrequest` 中。当请求进入 `loadbalancer` 模块时，直接从 `req` 中取出 `peer` 信息填充给 `nginx` 即可。

这样设计，逻辑更清晰，代码也更好组织，否则用户使用的时候，还得通过 `thread local` 变量传递 `peer` 地址。

在实现 `loadbalancer` 模块时，我们可以复用 `nginx` 内置的 `wrr` 模块。因为除了获取 `peer` 外，还有一些 `TLS` 相关的功能需要实现。

虽说 `proxy_certificate` 等配置是在 `location` 里面配的，但是 `nginx` 会将 `SSLContext` 缓存在 `peer` 的数据结构内部。复用 `wrr` 模块可以更方便的处理这些特殊场景。

到此，我们不但实现了基于 `nginx` 的 `http` 客户端，还顺带实现了动态 `upstream`。

libnginx 的 http 客户端可以复用 nginx 的 keepalive、TLS、连接池等能力，但是还有一个不小的缺陷：

很遗憾 nginx 官方社区至今都没有实现 upstream h2，社区的说法是“没有必要”，不过在 内网，http/1.x 也能够满足绝大多数需求了。

开源

目前 libnginx 上述功能均已实现，此外还利用了 **Swift** 的高级特性，把 libnginx 做了一层封装，让它看起来更像一个 http 框架。

欢迎大家尝试：

https://github.com/wkgcass/libnginx
github.com/wkgcass/libnginx

后续有时间会把 rust 的封装也实现出来（或者说有没有人愿意提个 pr ~）

还没有人送礼物，鼓励一下作者吧

所属专栏 · 2025-02-03 21:17 更新



wkgcass 的网络专栏
wkGCaSS
6 篇内容 · 344 赞同

订阅

最热内容 · 零依赖的软路由——使用 OpenVSwitch 改造家庭网络

发布于 2025-02-03 20:28 · 浙江



理性发言，友善互动

18 条评论

默认 最新



奥德修斯 不能理解工程意图。如果要用 Web 服务，直接部署 Nginx 即可，何必又套一层？这样做的目的是啥呢？
02-04 · 福建



wkGCaSS 作者 这是库啊，绝大多数程序都不能随便 fork。特别是 java/go 这种带运行时的。另外如果单独起一组 nginx 进程然后用 proxy_pass 转发才是“套一层”，用库的方式就只是几个函数调用而已
02-05 · 浙江



wkGCaSS 作者 网关场景，以及轻度使用 http server 的场景
02-06 · 江苏

展开其他 3 条回复 >



土土先生

写的真不错

02-07 · 广东

回复 1



12345

或者反过来，用swift给nginx开发个扩展

08-11 · 北京

回复 喜欢



dhepxxusnxuff

不赖

06-13 · 广东

回复 喜欢



卜史

这个想法是真牛💡

05-12 · 广西

回复 喜欢



谢慕安

不是有LUA模块的吗？好像JS的也有了。直接用脚本语言开发业务不香吗？

02-05 · 上海

回复 喜欢



wkGCaSS 作者

弱类型容错低，项目大了不敢改

02-06 · 江苏

回复 1



台湾省精神病院长

可以封装成Python格式吗

02-04 · 上海

回复 喜欢



wkGCaSS 作者

工程上没什么难度，只是我不太熟悉python的c接口，所以我目前搞不出来

02-04 · 浙江

回复 喜欢



代宣神谕之人

法律问题呢？

02-04 · 浙江

回复 喜欢



wkGCaSS 作者

没有法律问题，nginx开源协议很宽松

02-04 · 浙江

回复 1



harbor

王哥出品，必属精品

02-03 · 北京

回复 喜欢



重拳捶瓜娃

原来是老王

02-04 · 贵州

回复 喜欢



理性发言，友善互动